Московский государственный технический университет
имени Н.Э. Баумана

Кафедра «Системы обработки информации и управления»

Ю.Е. Гапанюк

УЧЕБНЫЕ МАТЕРИАЛЫ СЕМИНАРА №5
ВНЕДРЕНИЕ ЗАВИСИМОСТЕЙ

Учебные материалы по дисциплине
«Архитектура автоматизированных систем обработки информации и управления»
Профиль: «Архитектура программных систем»

Москва – 2026

Оглавление

1 ЦЕЛЬ ... 3

2 ПРОБЛЕМЫ СИЛЬНО СВЯЗАННОГО КОДА ... 4

2.1 Постановка задачи ... 4

2.2 Проблемы «наивной» реализации ... 5

2.2.1 Сценарий 1: требования изменились — нужно писать лог в файл ... 5

2.2.2 Сценарий 2: нужно несколько вариантов поведения одновременно ... 6

2.2.3 Сценарий 3: два разработчика не могут работать независимо ... 7

3 ОСНОВНЫЕ ОПРЕДЕЛЕНИЯ ВНЕДРЕНИЯ ЗАВИСИМОСТЕЙ ... 8

4 РЕАЛИЗАЦИЯ ВНЕДРЕНИЯ ЗАВИСИМОСТЕЙ ... 10

4.1 Реализация слабой связанности через механизм интерфейсов ... 10

4.2 Внедрение зависимости через конструктор ... 12

4.3 Внедрение зависимости через свойство ... 13

4.4 Внедрение зависимости через параметр метода ... 15

4.5 DI «для бедных» и точка сборки ... 16

5 ФРЕЙМВОРК MICROSOFT.EXTENSIONS.DEPENDENCYINJECTION ... 18

5.1 Когда Pure DI перестаёт быть удобным ... 18

5.2 Подключение пакета ... 19

5.3 Регистрация зависимостей и получение объектов ... 19

5.4 Жизненные циклы зависимостей ... 20

5.5 Проблема неправильного сочетания жизненных циклов ... 22

5.6 Рефакторинг точки сборки ... 23

6 АНТИПАТТЕРНЫ ВНЕДРЕНИЯ ЗАВИСИМОСТЕЙ ... 24

6.1 Антипаттерн «Тяга к контролю» (Control Freak) ... 24

6.2 Антипаттерн «Двойное внедрение» (Bastard Injection) ... 24

6.3 Антипаттерн «Локатор служб» (Service Locator) ... 25

6.4 Антипаттерн «Окружающий контекст» (Ambient Context) ... 27

7 КОНТРОЛЬНОЕ ЗАДАНИЕ ... 29

8 ВЫВОДЫ ... 31

1 Цель

Познакомиться с принципом внедрения зависимостей как инструментом проектирования слабосвязанных систем. Освоить ручное внедрение зависимостей и использование DI-контейнера Microsoft.Extensions.DependencyInjection. Научиться распознавать паттерны и антипаттерны внедрения зависимостей.

2 Проблемы сильно связанного кода

На предыдущих занятиях мы научились создавать иерархии классов, реализовывать абстрактные методы и интерфейсы. Казалось бы, этого достаточно для написания «правильного» кода — мы применяем классы, наследование, полиморфизм, как и положено в объектно-ориентированном языке.

Однако, как выясняется на практике, использование принципов ООП и объектно-ориентированного языка программирования еще не является «достаточным» условием проектирования хороших систем. Давайте рассмотрим «проблемную» ситуацию, с которой вы обязательно столкнётесь при разработке любого реального проекта.

2.1 Постановка задачи

Представьте, что вы разрабатываете сервис управления каталогом книг. Сервис должен добавлять книги и записывать информацию о произошедших событиях в журнал (лог) — это стандартная практика в большинстве приложений: и библиотечных системах, и интернет-магазинах, и банковском ПО.

Ниже показана «наивная» реализация, которая кажется вполне разумной:

```
/// <summary>
/// Логгер: выводит сообщения в консоль
/// </summary>
class NaiveConsoleLogger
{
    public void Log(string message)
    {
        Console.WriteLine($"[LOG] {message}");
    }
}

/// <summary>
/// Сервис каталога книг
/// </summary>
class NaiveBookCatalogService
```

```
{
    // Логгер создаётся прямо здесь, внутри класса
    // ЭТО ОСНОВНАЯ ПРОБЛЕМА
    private NaiveConsoleLogger _logger = new
NaiveConsoleLogger();

    public void AddBook(string title, string author)
    {
        // ... здесь была бы логика сохранения книги ...
        _logger.Log($"Добавлена книга: «{title}» — {author}");
    }

    public void RemoveBook(string title)
    {
        // ... здесь была бы логика удаления ...
        _logger.Log($"Удалена книга: «{title}»");
    }
}

// Основная программа
Console.WriteLine("=== Наивная реализация ===");
NaiveBookCatalogService service = new();
service.AddBook("Евгений Онегин", "Пушкин");
service.RemoveBook("Евгений Онегин");
```

Приведенный код компилируется и успешно работает. Какие проблемы он может вызывать?

2.2 Проблемы «наивной» реализации

Рассмотрим три сценария, которые неизбежно возникают в реальных проектах.

2.2.1 Сценарий 1: требования изменились — нужно писать лог в файл

Заказчик просит: «Нам нужно хранить журнал событий в файле, а не только выводить в консоль». Задача несложная — добавим класс NaiveFileLogger:

```
/// <summary>
/// Логгер: записывает сообщения в файл
/// </summary>
class NaiveFileLogger
```

```
{
    private string _filePath;

    public NaiveFileLogger(string filePath) { _filePath = filePath; }

    public void Log(string message)
    {
        File.AppendAllText(_filePath, $"[LOG] {message}\n");
    }
}
```

Но при этом необходимо изменить BookCatalogService — ведь поле _logger объявлено с конкретным типом ConsoleLogger. Нужно обязательно редактировать код класса NaiveBookCatalogService, хотя логирование не является прямой задачей этого класса:

```
class NaiveBookCatalogService
{
    // Чтобы сменить логгер — нужно поменять класс сервиса
    private NaiveFileLogger _logger = new NaiveFileLogger("catalog.log");
```

Если далее потребуется логгер, отправляющий сообщения в базу данных, необходимо снова редактировать BookCatalogService. Один и тот же класс, который должен заниматься только управлением каталогом, постоянно меняется по причинам, никак не связанным с его основной задачей.

2.2.2 Сценарий 2: нужно несколько вариантов поведения одновременно

В процессе разработки удобно видеть лог в консоли, в готовом продукте — писать в файл. Наиболее очевидным решением является добавление флага:

```
/// <summary>
/// Логирование с флагом
/// </summary>
class NaiveBookCatalogServiceLogFlag
{
    private bool _useFile;

    public NaiveBookCatalogServiceLogFlag(bool useFile = false)
                        { _useFile = useFile; }
```

```
    public void AddBook(string title, string author)
    {
        if (_useFile)
            File.AppendAllText("log.txt", $"Добавлена книга:
«{title}»\n");
        else
            Console.WriteLine($"[LOG] Добавлена книга: «{title}»");
    }

    // ... и в каждом методе тот же if/else ...
}
```

Это решение выглядит хуже исходного. При добавлении третьего варианта появится else if, при четвёртом — ещё один. Метод AddBook, который должен добавлять книгу, превращается в свалку кода, не имеющего отношения к его назначению.

Второй вариант — создать два класса: BookCatalogServiceConsole и BookCatalogServiceFile. Но это полное дублирование кода, причём любое исправление в логике работы с каталогом придётся вносить в оба класса одновременно.

Ни один из вариантов не является приемлемым.

2.2.3 Сценарий 3: два разработчика не могут работать независимо

Представьте, что над проектом работают два человека: один пишет BookCatalogService, другой — логгер. Но BookCatalogService напрямую зависит от конкретного класса ConsoleLogger. Пока второй разработчик не закончит свою часть, первый не может ни написать, ни проверить свой код. Работа выстраивается в очередь.

Теперь мысленно умножьте эту ситуацию на десяток сервисов и несколько команд — и получите то, что называют «спагетти-зависимостями»: любое изменение в одном месте системы вызывает цепную реакцию изменений по всему проекту.

Обратите внимание: все описанные проблемы возникли не из-за «неправильного» кода — мы использовали классы и поля в точности так, как предполагает язык C#.

Корень проблемы в одной строке:

```
private NaiveConsoleLogger _logger = new NaiveConsoleLogger();
```

Класс BookCatalogService сам создаёт свою зависимость и сам же её использует. Такой код принято называть сильно связанным (tightly coupled).

3 Основные определения внедрения зависимостей

Зависимость (dependency) — это любой объект, который необходим классу для выполнения его работы, но который класс не производит сам по себе. В нашем примере ConsoleLogger является зависимостью класса BookCatalogService.

Марк Симан в книге «Внедрение зависимостей в .NET» предлагает называть зависимость «соавтором» (collaborator): это объект, без которого класс не может выполнить свою работу. Зависимости есть у любого нетривиального класса — вопрос не в их наличии, а в том, кто несёт ответственность за их создание.

В «наивной» реализации BookCatalogService сам создаёт своего «соавтора» — оператором new. Именно это и является корнем всех описанных проблем.

Для решения описанных проблем в разработке программного обеспечения сформировались два тесно связанных понятия.

Инверсия управления (Inversion of Control, IoC) — архитектурный принцип, согласно которому класс не управляет созданием своих зависимостей самостоятельно: эта ответственность передаётся внешнему коду. Слово «инверсия» отражает переворот привычного порядка вещей —

вместо того чтобы класс сам создавал нужные ему объекты, кто-то извне передаёт ему уже готовые.

Внедрение зависимостей (Dependency Injection, DI) — конкретный способ реализации принципа инверсии управления. Вместо создания зависимостей оператором new внутри класса их передают классу извне — через конструктор, свойство или параметр метода.

Соотношение этих понятий простое: IoC — это принцип, DI — способ его реализации. IoC описывает что нужно сделать (снять с класса ответственность за создание зависимостей), а DI описывает как это сделать (передать зависимости снаружи).

Важно понимать, что DI — не единственный способ реализации IoC. Существует, например, паттерн «Service Locator», при котором класс сам запрашивает зависимости у специального реестра объектов — это тоже инверсия управления, но, как будет показано далее, данный подход является антипаттерном. Поэтому в контексте данного семинара, говоря «инверсия управления», мы будем всегда иметь в виду реализацию именно через внедрение зависимостей.

Мы написали объектно-ориентированный код, использовали классы, поля и методы. Но этого оказалось недостаточно для получения гибкого и легко поддерживаемого результата. Объектно-ориентированное программирование — это инструментарий: классы, наследование, полиморфизм. Внедрение зависимостей — это принцип применения этого инструментария таким образом, чтобы классы оставались независимыми друг от друга в части создания объектов.

В таблице ниже показано, что изменится при переходе от сильно связанного кода к слабосвязанному:

	Сильно связанный код	**Слабосвязанный код (DI)**
Кто создаёт зависимость	Сам класс (`new ConsoleLogger()`)	Внешний код
Смена реализации логгера	Правка исходного кода сервиса	Передача другого объекта снаружи
Зависит ли сервис от `ConsoleLogger`	Да, «сильная» связь	Нет
Независимая разработка двух классов	Невозможна	Возможна

Возникает закономерный вопрос: как именно передать «другой объект снаружи», если у FileLogger и ConsoleLogger нет ничего общего с точки зрения системы типов C#? Ответ на этот вопрос — и есть тема следующей части: собственные интерфейсы как инструмент слабой связанности.

4 Реализация внедрения зависимостей

В предыдущей части мы выяснили, что корень проблемы сильно связанного кода — строка new ConsoleLogger() внутри BookCatalogService. Класс сам создаёт зависимость и сам же её использует. Теперь разберём, как это исправить.

4.1 Реализация слабой связанности через механизм интерфейсов

Интерфейс определяет контракт — набор методов, которые обязуется реализовать любой класс, подписавшийся под этим контрактом. Именно это свойство интерфейсов и является ключом к решению проблемы.

Вместо того чтобы BookCatalogService зависел от конкретного класса ConsoleLogger, сделаем так, чтобы он зависел от интерфейса — абстракции, которая описывает только то, что сервис хочет от логгера:

```
/// <summary>
/// Контракт для записи журнала событий.
/// Интерфейс определяется исходя из потребностей того,
/// кто его использует, а не того, кто его реализует.
/// </summary>
interface ILogger
{
    void Log(string message);
}
```

Реализуем этот интерфейс в конкретных классах-логгерах:

```
/// <summary>
/// Логгер: выводит сообщения в консоль
/// </summary>
class ConsoleLogger : ILogger
{
    public void Log(string message)
    {
        Console.WriteLine($"[LOG] {message}");
    }
}

/// <summary>
/// Логгер: записывает сообщения в файл
/// </summary>
class FileLogger : ILogger
{
    private string _filePath;

    public FileLogger(string filePath) { _filePath = filePath; }

    public void Log(string message)
    {
        File.AppendAllText(_filePath, $"[LOG] {message}\n");
    }
}
```

Марк Симан в своей книге подчеркивает принципиальный момент: интерфейс ILogger принадлежит тому, кто его использует — BookCatalogService, а не тому, кто его реализует. Мы определили контракт исходя из потребностей сервиса (один метод Log), а не исходя из

возможностей конкретного логгера. ConsoleLogger, FileLogger и любой новый логгер, который мы создадим в будущем, для BookCatalogService совершенно неразличимы — он видит только тип ILogger.

4.2 Внедрение зависимости через конструктор

Теперь перепишем BookCatalogService так, чтобы он получал зависимость снаружи. Наиболее распространённый и рекомендуемый способ — внедрение через конструктор (constructor injection):

```
/// <summary>
/// Сервис каталога книг.
/// Зависит от абстракции ILogger, а не от конкретного класса.
/// </summary>
class BookCatalogService_DI_Constructor
{
    private ILogger _logger;

    /// <summary>
    /// Зависимость передаётся снаружи — класс её не создаёт.
    /// Зависимость внедряется через конструктор
    /// </summary>
    public BookCatalogService_DI_Constructor(ILogger logger)
    {
        _logger = logger;
    }

    public void AddBook(string title, string author)
    {
        // ... логика сохранения книги ...
        _logger.Log($"Добавлена книга: «{title}» — {author}");
    }

    public void RemoveBook(string title)
    {
        // ... логика удаления ...
        _logger.Log($"Удалена книга: «{title}»");
    }
}
```

Посмотрим, что изменилось: поле _logger теперь имеет тип ILogger, а не ConsoleLogger; строка new ConsoleLogger() полностью исчезла из класса; конструктор принимает ILogger как параметр. Попробуем использовать переработанный сервис:

```
Console.WriteLine("=== Внедрение через конструктор ===");
// Передаём ConsoleLogger
BookCatalogService_DI_Constructor s1 = new
BookCatalogService_DI_Constructor(new ConsoleLogger());
s1.AddBook("Евгений Онегин", "Пушкин");

// Передаём FileLogger — код BookCatalogService не трогаем!
BookCatalogService_DI_Constructor s2 = new
BookCatalogService_DI_Constructor(new FileLogger("log.txt"));
s2.AddBook("Сборник стихотворений", "Пушкин");
```

Основным преимуществом решения является то, что смена логгера стала возможна без изменения кода BookCatalogService — нужно лишь передать другой вид логгера в конструктор при создании объекта класса. Именно это и называется слабой связанностью (loose coupling): классы взаимодействуют через контракты-интерфейсы, а не через конкретные реализации.

Внедрение через конструктор является предпочтительным способом по следующим причинам:

1. Зависимость явная и обязательная: нельзя создать объект BookCatalogService, не предоставив ему логгер — компилятор не позволит.
2. Зависимость видна в публичном API класса: просто взглянув на сигнатуру конструктора, разработчик сразу понимает, что именно нужно сервису для работы.
3. Объект полностью готов к работе сразу после создания — не нужно вызывать дополнительные методы инициализации.

4.3 Внедрение зависимости через свойство

Иногда зависимость является необязательной: класс умеет работать и без неё, используя разумное поведение по умолчанию. Для таких случаев применяется внедрение через свойство (property injection).

Представим, что логирование в нашем сервисе необязательно: в «тихом» режиме сервис просто ничего не записывает. Сначала создадим специальный «пустой» логгер:

```
/// <summary>
/// Логгер-заглушка: ничего не делает.
/// Применяется как безопасное значение по умолчанию вместо null.
/// </summary>
class NullLogger : ILogger
{
    // Метод ничего не делает
    public void Log(string message) { }
}
```

Перепишем сервис с внедрением через свойство:

```
/// <summary>
/// Сервис каталога книг с необязательным логгером.
/// </summary>
class BookCatalogService_DI_Property
{
    /// <summary>
    /// Необязательная зависимость.
    /// Если не задана — используется NullLogger (ничего не
делает).
    /// </summary>
    public ILogger Logger { get; set; } = new NullLogger();

    public void AddBook(string title, string author)
    {
        // ... логика сохранения книги ...
        Logger.Log($"Добавлена книга: «{title}» — {author}");
    }

    public void RemoveBook(string title)
    {
        // ... логика удаления ...
        Logger.Log($"Удалена книга: «{title}»");
    }
}
```

Пример использования:

```
Console.WriteLine("=== Внедрение через свойство ===");
BookCatalogService_DI_Property s3 = new();

// Без логгера — NullLogger работает молча, никакого вывода
s3.AddBook("Евгений Онегин", "Пушкин");

// Подключаем логгер через свойство
s3.Logger = new ConsoleLogger();
```

```
s3.AddBook("Сборник стихотворений", "Пушкин"); // теперь событие
пишется в консоль
```

Обратите внимание на использование NullLogger вместо null. Это стандартный приём — паттерн «Нулевой объект» (Null Object): вместо того чтобы перед каждым вызовом Logger.Log(...) проверять, не равен ли Logger значению null, мы подставляем объект с пустой реализацией. Исключение NullReferenceException исключено в принципе, а код остаётся чистым.

Внедрение через свойство имеет существенный недостаток по сравнению с конструктором: объект можно создать и начать использовать, не зная о том, что у него есть настраиваемые зависимости. Именно поэтому данный способ применяется только для необязательных зависимостей, когда поведение по умолчанию разумно и безопасно.

4.4 Внедрение зависимости через параметр метода

Третий способ применяется в ситуации, когда зависимость нужна только для одного конкретного вызова, причём она может меняться от вызова к вызову. В этом случае зависимость передаётся как параметр метода — внедрение через метод (method injection):

```
/// <summary>
/// Сервис каталога книг.
/// Логгер передаётся отдельно для каждой операции.
/// </summary>
class BookCatalogService_DI_Method
{
    // Поля нет — логгер приходит с каждым вызовом
    public void AddBook(string title, string author, ILogger
logger)
    {
        // ... логика сохранения книги ...
        logger.Log($"Добавлена книга: «{title}» — {author}");
    }
}
```

Пример использования:

```
Console.WriteLine("=== Внедрение через параметр метода ===");
BookCatalogService_DI_Method s4 = new
BookCatalogService_DI_Method();
// Разные операции могут логироваться по-разному
s4.AddBook("Евгений Онегин", "Пушкин", new ConsoleLogger());
s4.AddBook("Сборник стихотворений", "Пушкин", new
FileLogger("audit.log"));
```

Внедрение через метод применяется значительно реже двух предыдущих способов.

4.5 DI «для бедных» и точка сборки

Мы научились принимать зависимости. Но кто их создаёт? Где должна вызываться строка new ConsoleLogger()?

Ответ на этот вопрос — один из важнейших принципов внедрения зависимостей: всё создание объектов и сборка графа зависимостей должны быть сосредоточены в одном месте, расположенном как можно ближе к точке входа в приложение. Это место называется точкой сборки (Composition Root). В консольном приложении точкой сборки как правило является метод Main. Вся остальная программа понятия не имеет о конкретных реализациях — только точка сборки знает, какие классы выбраны.

Добавим вторую зависимость — хранилище книг — чтобы увидеть, как точка сборки управляет несколькими зависимостями одновременно:

```
/// <summary>
/// Контракт для хранения книг
/// </summary>
interface IBookStorage
{
    void Save(string title, string author);
}

/// <summary>
/// Хранение книг в оперативной памяти
/// </summary>
class InMemoryBookStorage : IBookStorage
{
    private ILogger _logger;
    private List<string> _books = new();

    public InMemoryBookStorage(ILogger logger) { _logger = logger; }
```

```
    public void Save(string title, string author)
    {
        _books.Add($"«{title}» — {author}");
        _logger.Log($" [STORAGE] Сохранено: «{title}»");
    }
}
```

Обновим BookCatalogService, чтобы он принимал обе зависимости через конструктор:

```
/// <summary>
/// Сервис каталога книг.
/// Зависит только от интерфейсов — ни одного конкретного класса.
/// </summary>
class BookCatalogService
{
    private ILogger _logger;
    private IBookStorage _storage;

    public BookCatalogService(ILogger logger, IBookStorage storage)
    {
        _logger = logger;
        _storage = storage;
    }

    public void AddBook(string title, string author)
    {
        _storage.Save(title, author);
        _logger.Log($"Добавлена книга: «{title}» — {author}");
    }

    public void RemoveBook(string title)
    {
        _logger.Log($"Удалена книга: «{title}»");
    }
}
```

Обратим внимание: BookCatalogService не содержит ни одного имени конкретного класса — только ILogger и IBookStorage. Он буквально не знает, кто выполняет за него эту работу.

Теперь — точка сборки в методе Main. Марк Симан называет такой подход DI «для бедных» (Poor Man's DI) или чистым DI (Pure DI): зависимости собираются вручную, без использования специальных фреймворков.

```
Console.WriteLine("=== Точка сборки ===");
ILogger logger = new ConsoleLogger();
IBookStorage storage = new InMemoryBookStorage(logger);
```

```
BookCatalogService s5 = new BookCatalogService(logger, storage);
s5.AddBook("Евгений Онегин", "Пушкин");
s5.AddBook("Сборник стихотворений", "Пушкин");
```

Чтобы поменять логгер на файловый, достаточно изменить **одну строку** в точке сборки.

Обратите внимание, что в InMemoryBookStorage и BookCatalogService можно передать разные логгеры, но, как правило, смысл именно в том, чтобы в разных частях системы использовались общие объекты классов (в данном случае объект класса логгера).

Pure DI — полноценный рабочий подход, и многие опытные разработчики предпочитают его для небольших проектов именно потому, что он не требует дополнительных библиотек, а весь граф объектов виден прямо в Main.

Но в случае больших проектов более удобным становится использование специализированных DI-фреймворков.

5 Фреймворк Microsoft.Extensions.DependencyInjection

5.1 Когда Pure DI перестаёт быть удобным

Pure DI удобен, пока дерево объектов небольшое. Но представьте, что проект вырос: в нём несколько десятков сервисов, некоторые имеют по пять-шесть зависимостей, а те, в свою очередь, тоже зависят от других объектов. Точка сборки превращается в сотни строк ручного new, и любое добавление зависимости требует ручного обновления этого кода.

Именно для решения этой проблемы существуют DI-контейнеры (DI containers). Контейнер — это объект, которому вы сообщаете, какие интерфейсы каким конкретным классам соответствуют, а он сам строит граф зависимостей и создаёт нужные объекты в правильном порядке.

Точка сборки при этом остаётся в одном месте, но вместо ручного new вы регистрируете типы.

5.2 Подключение пакета

Для работы с DI-контейнером потребуется пакет Microsoft.Extensions.DependencyInjection. Это стандартный контейнер платформы .NET, используемый в ASP.NET Core и большинстве современных .NET-приложений. Он прост в освоении и не требует XML-конфигурации или специальных атрибутов.

Для подключения пакета выполним в терминале следующую команду в папке проекта:

dotnet add package Microsoft.Extensions.DependencyInjection

или в консоли диспетчера пакетов:

Install-Package Microsoft.Extensions.DependencyInjection

После этого добавим в начало файла Program.cs директиву:

using Microsoft.Extensions.DependencyInjection;

5.3 Регистрация зависимостей и получение объектов

Работа с контейнером состоит из двух этапов: регистрации (мы сообщаем контейнеру, что и как создавать) и разрешения (мы просим контейнер создать нужный объект).

Рассмотрим базовый API на примере наших классов:

```
// Шаг 1 — создаём коллекцию описаний зависимостей
ServiceCollection services = new ServiceCollection();

// Шаг 2 — регистрация: говорим контейнеру,
// какой конкретный класс использовать для каждого интерфейса
services.AddSingleton<ILogger, ConsoleLogger>();
services.AddSingleton<IBookStorage, InMemoryBookStorage>();
services.AddTransient<BookCatalogService>();

// Шаг 3 — строим контейнер (после этого регистрация закрыта)
ServiceProvider provider = services.BuildServiceProvider();
```

```
// Шаг 4 — разрешение: просим контейнер создать нужный объект
BookCatalogService s7 =
provider.GetRequiredService<BookCatalogService>();
// Работаем с объектом как обычно
s7.AddBook("Евгений Онегин", "Пушкин");
```

Обратите внимание: мы нигде не написали new ConsoleLogger() или new InMemoryBookStorage(...) явно — контейнер сам разобрался, что BookCatalogService нуждается в ILogger и IBookStorage, что InMemoryBookStorage нуждается в ILogger, и создал всё в правильном порядке. При этом код классов не изменился.

Метод GetRequiredService<T> возвращает запрошенный объект или генерирует исключение, если тип не зарегистрирован. Существует также метод GetService<T>, который в случае отсутствия регистрации возвращает null — однако GetRequiredService<T> предпочтительнее: ошибка конфигурации немедленно проявляется как исключение, а не как загадочный NullReferenceException где-то внутри программы.

5.4 Жизненные циклы зависимостей

При регистрации зависимости необходимо указать её жизненный цикл (lifetime) — правило, по которому контейнер решает, создавать ли новый объект при каждом запросе или возвращать уже созданный.

В Microsoft.Extensions.DependencyInjection предусмотрены три жизненных цикла.

1. Transient — новый объект при каждом запросе.

Transient подходит для лёгких объектов, не хранящих состояния, когда важно, чтобы каждый кто запрашивает объект получил независимый экземпляр.

2. Singleton — один объект на всё время работы приложения.

Контейнер создаёт экземпляр ConsoleLogger один раз и возвращает его всем, кто запрашивает ILogger, — независимо от того, сколько раз и кто именно делает запрос.

Именно поэтому в нашем примере правильно регистрировать ILogger как Singleton: и BookCatalogService, и InMemoryBookStorage получат один и тот же экземпляр логгера, а не независимые копии. Singleton подходит для объектов, которые дороги в создании или которые по смыслу должны быть общими — логгер, кеш, конфигурация.

3. Scoped — один объект в рамках области видимости.

Scoped занимает промежуточное положение между Transient и Singleton. В рамках одной области видимости (scope) контейнер возвращает один и тот же экземпляр, но при создании новой области — создаётся новый объект. В консольном приложении области создаются вручную:

```
services.AddScoped<IBookStorage, InMemoryBookStorage>();
ServiceProvider provider = services.BuildServiceProvider(
    new ServiceProviderOptions { ValidateOnBuild = true }
);

using (IServiceScope scope1 = provider.CreateScope())
{
    IBookStorage s1 =
scope1.ServiceProvider.GetRequiredService<IBookStorage>();
    IBookStorage s2 =
scope1.ServiceProvider.GetRequiredService<IBookStorage>();

    // true — в рамках одной области это один объект
    Console.WriteLine(object.ReferenceEquals(s1, s2));
}

using (IServiceScope scope2 = provider.CreateScope())
{
    // Новая область — новый независимый экземпляр
    IBookStorage s3 =
scope2.ServiceProvider.GetRequiredService<IBookStorage>();
}
```

Scoped наиболее востребован в веб-приложениях, где одна область соответствует одному HTTP-запросу: все компоненты, обрабатывающие

один запрос, работают с одним экземпляром зависимости, а для следующего запроса создаётся новый.

5.5 Проблема неправильного сочетания жизненных циклов

Выбор жизненного цикла — ответственное проектное решение. Наиболее распространённая ошибка — внедрение короткоживущей зависимости в долгоживущий объект:

```
// BookCatalogService – Singleton, IBookStorage – Transient
services.AddSingleton<BookCatalogService>();
services.AddTransient<IBookStorage, InMemoryBookStorage>();
services.AddSingleton<ILogger, ConsoleLogger>();
```

Singleton-объект BookCatalogService создаётся один раз и хранит ссылку на IBookStorage, полученную в момент создания. Несмотря на то, что IBookStorage зарегистрирован как Transient, BookCatalogService будет всегда использовать один и тот же экземпляр хранилища — тот, что был передан при его создании. Transient-регистрация фактически перестаёт работать.

Контейнер Microsoft.Extensions.DependencyInjection умеет обнаруживать такие ситуации и генерировать исключение InvalidOperationException. Это поведение называется проверкой при сборке и включается следующим образом:

```
// Проверка корректности всех регистраций при вызове
BuildServiceProvider
ServiceProvider provider = services.BuildServiceProvider(
    new ServiceProviderOptions { ValidateOnBuild = true }
);
```

При неверном сочетании жизненных циклов программа не запустится и сразу сообщит об ошибке — это значительно лучше, чем обнаружить проблему в процессе работы.

Практическое правило: короткоживущий объект может зависеть от долгоживущего, но не наоборот. Transient может зависеть от Singleton, Singleton не должен зависеть от Transient.

В таблице приведены рекомендации по выбору жизненного цикла:

Жизненный цикл	**Когда использовать**	**Пример**
Singleton	Объект дорог в создании или должен быть общим	Логгер, конфигурация, кеш
Scoped	Один экземпляр на операцию или HTTP-запрос	Контекст базы данных
Transient	Лёгкие объекты без состояния	Сервисы бизнес-логики

5.6 Рефакторинг точки сборки

Рассмотрим финальную точку сборки на основе фреймворка.

Важно, что при использовании фреймворка код классов и интерфейсов не меняется. Единственное, что изменяется, — точка сборки в Main.

```
Console.WriteLine("=== Точка сборки с фреймворком ===");
// Шаг 1 — создаём коллекцию описаний зависимостей
ServiceCollection services = new ServiceCollection();

// Шаг 2 — регистрация: говорим контейнеру,
// какой конкретный класс использовать для каждого интерфейса
services.AddSingleton<ILogger, ConsoleLogger>();
services.AddSingleton<IBookStorage, InMemoryBookStorage>();
services.AddTransient<BookCatalogService>();

// Шаг 3 — строим контейнер (после этого регистрация закрыта)
ServiceProvider provider = services.BuildServiceProvider(
    new ServiceProviderOptions { ValidateOnBuild = true }
);

// Шаг 4 — разрешение: просим контейнер создать нужный объект
```

```
// Контейнер сам строит граф: BookCatalogService → ILogger,
IBookStorage
//                                InMemoryBookStorage → ILogger
BookCatalogService s7 =
provider.GetRequiredService<BookCatalogService>();
// Работаем с объектом как обычно
s7.AddBook("Евгений Онегин", "Пушкин");
```

6 Антипаттерны внедрения зависимостей

Антипаттерн — это решение, которое на первый взгляд кажется разумным, но на практике воспроизводит те же проблемы, что и сильно связанный код: скрытые зависимости, невозможность смены реализации, неожиданное поведение при изменениях.

Каждый из описанных ниже антипаттернов рассматривается по схеме: плохой код → симптомы → исправленный код.

6.1 Антипаттерн «Тяга к контролю» (Control Freak)

«Тяга к контролю» — ситуация, когда класс сам создаёт свои зависимости оператором new, вместо того чтобы получать их извне. Это именно та проблема, с которой мы начинали семинар. «Тяга к контролю» — это название антипаттерна из книги Марка Симана.

6.2 Антипаттерн «Двойное внедрение» (Bastard Injection)

«Двойное внедрение» — это попытка совместить внедрение через конструктор с созданием зависимости «по умолчанию». Класс предоставляет два конструктора: один принимает зависимость извне, другой — без параметров — создаёт её сам.

Пример плохого кода:

```
class BookCatalogService_Anti_BI
{
    private ILogger _logger;

    /// <summary>
    /// Конструктор «для удобства»: создаёт логгер
самостоятельно.
```

```
    /// Кажется полезным — на деле скрывает зависимость.
    /// </summary>
    public BookCatalogService_Anti_BI() : this(new
ConsoleLogger()) { }

    /// <summary>
    /// Конструктор с внедрением: принимает логгер снаружи.
    /// </summary>
    public BookCatalogService_Anti_BI(ILogger logger)
    {
        _logger = logger;
    }

    public void AddBook(string title, string author)
    {
        _logger.Log($"Добавлена книга: «{title}» — {author}");
    }
}
```

Симптомы:

- Конструктор без параметров создаёт иллюзию, что класс независим, — но внутри него жёстко прописан ConsoleLogger.
- Снаружи невозможно понять, какой логгер будет использован при вызове new BookCatalogService().
- DI-контейнер может неожиданно выбрать конструктор без параметров и проигнорировать зарегистрированный ILogger.

Решение:

1. Убрать конструктор без параметров.
2. Если нужно поведение «по умолчанию» то лучше передать NullLogger в точке сборки явно. Выбор реализации по умолчанию остаётся там, где он должен быть: в точке сборки, а не скрытым внутри класса.

6.3 Антипаттерн «Локатор служб» (Service Locator)

«Локатор служб» — это глобальный реестр, из которого классы сами запрашивают нужные зависимости. На первый взгляд выглядит как внедрение зависимостей — ведь класс не создаёт объекты оператором new. Но это обманчивое сходство.

Пример плохого кода:

```
/// <summary>
/// Глобальный реестр зависимостей — антипаттерн Service
Locator.
/// </summary>
static class ServiceLocator
{
    private static ServiceProvider _provider;

    public static void Init(ServiceProvider provider) {
_provider = provider; }

    public static T Get<T>() =>
_provider.GetRequiredService<T>();
}

class BookCatalogService_Anti_SL
{
    public void AddBook(string title, string author)
    {
        // Зависимости запрашиваются внутри метода — скрытая
связанность
        ILogger logger = ServiceLocator.Get<ILogger>();
        IBookStorage storage =
ServiceLocator.Get<IBookStorage>();

        storage.Save(title, author);
        logger.Log($"Добавлена книга: «{title}» — {author}");
    }
}

Пример использования:

ServiceCollection services = new ServiceCollection();
services.AddSingleton<ILogger, ConsoleLogger>();
services.AddSingleton<IBookStorage, InMemoryBookStorage>();
ServiceLocator.Init(services.BuildServiceProvider());

// Создаём сервис без каких-либо зависимостей — выглядит просто,
// но скрывает реальную картину зависимостей
BookCatalogService service = new BookCatalogService();
service.AddBook("Евгений Онегин", "Пушкин");
```

Симптомы:

1. Зависимости BookCatalogService невидимы: взглянув на new BookCatalogService(), невозможно понять, что сервису нужны ILogger и IBookStorage.
2. Класс по-прежнему тесно связан с конкретным механизмом — просто вместо new ConsoleLogger() теперь написано ServiceLocator.Get<ILogger>().
3. Если ServiceLocator не инициализирован или ILogger не зарегистрирован, программа упадёт в момент вызова AddBook — не при создании объекта и не при старте приложения.

Именно поэтому Service Locator, несмотря на то что технически является реализацией IoC, считается антипаттерном: он сохраняет главные недостатки сильно связанного кода, лишь заменяя new на вызов реестра.

Решение: вернуть классический подход с зависимостями через конструктор.

Обратите внимание: DI-контейнер сам по себе не является Service Locator. Разница принципиальная: контейнер используется только в точке сборки и невидим остальному коду приложения. Service Locator — это когда контейнер (или аналогичный реестр) используется внутри прикладных классов для получения зависимостей.

6.4 Антипаттерн «Окружающий контекст» (Ambient Context)

«Окружающий контекст» — это глобальное статическое свойство, через которое классы получают зависимость, не запрашивая её явно через конструктор.

Плохой код:

```
/// <summary>
/// Глобальный контекст логгера — антипаттерн Ambient Context.
/// </summary>
static class LoggerContext
{
```

```
    // Глобальное состояние, доступное для изменения из любого
места программы
    public static ILogger Current { get; set; } = new
NullLogger();
}

class BookCatalogService_LC
{
    // Конструктор пуст — зависимость скрыта
    public void AddBook(string title, string author)
    {
        // Зависимость берётся из глобального контекста —
неявная связь
        LoggerContext.Current.Log($"Добавлена книга: «{title}» —
{author}");
    }

    public void RemoveBook(string title)
    {
        LoggerContext.Current.Log($"Удалена книга: «{title}»");
    }
}
```

Симптомы:

1. Зависимость от логгера невидима: конструктор BookCatalogService пуст, но класс тайно обращается к глобальному состоянию.
2. Значение LoggerContext.Current может быть изменено из любого места программы в любой момент — найти источник неожиданного поведения крайне сложно.
3. В многопоточном коде разные потоки могут одновременно читать и изменять LoggerContext.Current, что приводит к трудновоспроизводимым ошибкам.

Необходимо сделать замечание: окружающий контекст иногда оправдан для инфраструктурных зависимостей уровня среды выполнения, которые не являются частью бизнес-логики. Пример из стандартной библиотеки .NET: TimeZoneInfo.Local (часовой пояс). Их использование в виде глобального контекста приемлемо именно потому, что они принадлежат среде исполнения, а не конкретному приложению. Для зависимостей прикладного уровня — логгеров, хранилищ, сервисов — следует всегда использовать явное внедрение зависимостей.

7 Контрольное задание

В коде ниже намеренно допущены четыре нарушения из разобранных в этой части. Найдите каждый антипаттерн, назовите его и исправьте код.

```
static class AppServices
{
    public static ServiceProvider Provider { get; set; }
    public static T Get<T>() => Provider.GetRequiredService<T>();
}

static class CurrentLogger
{
    public static ILogger Instance { get; set; } = new NullLogger();
}

class BookCatalogService
{
    private IBookStorage _storage;

    public BookCatalogService()
        : this(new InMemoryBookStorage(new ConsoleLogger())) { }

    public BookCatalogService(IBookStorage storage)
    {
        _storage = storage;
    }

    public void AddBook(string title, string author)
    {
        ILogger logger = AppServices.Get<ILogger>();
        _storage.Save(title, author);
        logger.Log($"Добавлена книга: «{title}» — {author}");
    }

    public void RemoveBook(string title)
    {
        CurrentLogger.Instance.Log($"Удалена книга: «{title}»");
    }
}
```

РЕШЕНИЕ:

Разберём каждое нарушение:

1. new InMemoryBookStorage(new ConsoleLogger()) в конструкторе без параметров — Control Freak: класс сам создаёт зависимость оператором new.
2. Два конструктора, один из которых создаёт зависимость самостоятельно — Bastard Injection: зависимость одновременно обязательна и необязательна.
3. AppServices.Get<ILogger>() внутри метода AddBook — Service Locator: зависимость запрашивается из глобального реестра внутри прикладного класса.
4. CurrentLogger.Instance.Log(...) внутри метода RemoveBook — Ambient Context: зависимость берётся из глобального статического состояния.

8 Выводы

В данном семинаре мы прошли путь от сильно связанного кода — где один класс жёстко зависит от другого через оператор new — до слабосвязанной архитектуры, в которой классы взаимодействуют через интерфейсы и не знают о конкретных реализациях своих зависимостей.

Перечислим ключевые выводы.

- Объектно-ориентированное программирование — необходимое, но недостаточное условие хорошо поддерживаемого кода. Классы, наследование и полиморфизм дают инструментарий, но не гарантируют слабой связанности. Внедрение зависимостей — это принцип применения этого инструментария.
- Инверсия управления (IoC) и внедрение зависимостей (DI) — связанные, но разные понятия. IoC — архитектурный принцип: ответственность за создание зависимостей передаётся внешнему коду. DI — конкретный способ его реализации: зависимости передаются через конструктор, свойство или параметр метода. DI является предпочтительной формой IoC.
- Основной способ внедрения — через конструктор. Он делает зависимости явными, обязательными и видимыми в публичном API класса. Свойство и параметр метода применяются в специфических ситуациях.
- Точка сборки — единственное место в приложении, где создаются конкретные объекты. Вся остальная программа работает только через интерфейсы. Это ключевой структурный элемент любого приложения, использующего DI.
- Pure DI («для бедных») — полноценный подход, в котором граф зависимостей собирается вручную в точке сборки. Не требует дополнительных библиотек и подходит для проектов с небольшим числом зависимостей.

- DI-контейнер автоматизирует построение графа зависимостей. Стандартный контейнер .NET — Microsoft.Extensions.DependencyInjection — прост в освоении и достаточен для большинства задач. Выбор жизненного цикла (Singleton, Transient, Scoped) является важным проектным решением.